Workflow Design Document

Local AI Software Team

1. Document Overview

Product Name

Local AI Software Team

Document Type

Workflow Design Document

Purpose

This document defines the main workflows of Local AI Software Team.

It explains how a rough requirement moves through different AI roles and product stages until it becomes structured software delivery output.

This document focuses on:

- workflow stages
- agent responsibilities
- user approval points
- generated artifacts
- run statuses
- failure handling
- workflow modes
- end-to-end user journeys

This document does not focus on low-level technical implementation.

---

2. Workflow Philosophy

Local AI Software Team should behave like a structured software team, not like a single AI chat.

A normal AI coding tool often works like this:

```
User prompt → AI writes code
```

Local AI Software Team should work like this:

```
Rough requirement
→ Requirement clarification
→ Scope definition
→ Technical planning
→ Task breakdown
→ Test planning
→ User approval
→ Implementation
→ Test execution
→ Review
→ Traceability
→ Final report
```

The workflow should prioritize:

1. Clarity before coding.
2. Human approval before risky actions.
3. Traceability from requirement to output.
4. Local-first execution.
5. Reusable generated artifacts.
6. Controlled AI autonomy.
7. Reviewable and auditable results.

---

3. Workflow Principles

3.1 Requirement-first

The product should never jump directly from a vague requirement to code.

Before coding, the system should generate:

- clarified requirement
- business rules
- acceptance criteria
- assumptions
- open questions
- technical design
- task breakdown
- test matrix

3.2 Human approval by default

The system should require user approval before:

- code generation
- external tool update
- commit
- pull request creation

- risky command execution
- rollback
- publishing final report externally

3.3 Local-first execution

The system should run locally and save all workflow artifacts locally.

External integrations are optional.

3.4 Agent role separation

Different AI roles should have different responsibilities.

The system should avoid one generic AI response that does everything.

3.5 Review before delivery

Every implementation workflow should include:

- test result
- code review
- requirement coverage check
- final report

3.6 Traceability matters

The product should map:

```
Requirement → Acceptance Criteria → Task → Code → Test → Result
```

Traceability is a core differentiator.

---

4. Workflow Actors

4.1 Human User

The user is the final decision-maker.

The user can:

- input requirement
- approve or reject generated docs
- edit assumptions
- answer clarification questions
- approve implementation
- pause workflow
- cancel workflow

- review final result
- export artifacts

4.2 BA Agent

The Business Analyst Agent clarifies vague requirements.

Responsibilities:

- understand raw requirement
- extract business rules
- detect missing information
- ask clarification questions
- define acceptance criteria
- list assumptions
- identify out-of-scope items

Main outputs:

- clarified requirement
- business rules
- open questions
- assumptions
- acceptance criteria

4.3 Product Owner Agent

The Product Owner Agent defines scope and product value.

Responsibilities:

- define product goal
- identify MVP scope
- prioritize requirements
- define success criteria
- mark out-of-scope items

Main outputs:

- product objective
- MVP scope
- priority list
- out-of-scope list
- success criteria

4.4 Project Manager Agent

The Project Manager Agent turns requirements into work items.

Responsibilities:

- create epic/story/subtask structure
- define task dependencies
- estimate complexity
- define definition of done
- prepare Jira-ready output

Main outputs:

- epic
- user stories
- subtasks
- dependencies
- estimates
- definition of done

4.5 Architect Agent

The Architect Agent designs the technical solution.

Responsibilities:

- analyze requirement
- identify affected modules
- design APIs
- design database changes
- define service flow
- identify risks
- create implementation plan

Main outputs:

- technical design
- API design
- database design
- service flow
- risk list
- implementation plan

4.6 Developer Agent

The Developer Agent implements approved plans using a selected AI coding tool.

Responsibilities:

- follow technical design
- make minimal code changes
- follow project conventions
- write or update tests
- avoid unrelated changes

- explain changed files

Main outputs:

- code changes
- changed file list
- implementation notes
- patch/diff

4.7 QA Agent

The QA Agent validates the requirement and implementation.

Responsibilities:

- create test matrix
- map test cases to acceptance criteria
- analyze test failures
- identify missing edge cases
- verify expected behavior

Main outputs:

- test matrix
- test scenarios
- test result summary
- QA findings

4.8 Code Reviewer Agent

The Code Reviewer Agent checks implementation quality.

Responsibilities:

- review code diff
- check requirement coverage
- detect unrelated changes
- check maintainability
- check test quality
- identify required fixes

Main outputs:

- review report
- approval status
- required fixes

4.9 Security Reviewer Agent

The Security Reviewer Agent checks safety and security risks.

Responsibilities:

- check auth/authz
- check input validation
- detect secret leakage
- check unsafe file edits
- check risky commands
- detect sensitive logging

Main outputs:

- security review
- risk list
- required security fixes

4.10 Reporter Agent

The Reporter Agent summarizes the final delivery.

Responsibilities:

- summarize requirement
- summarize plan
- summarize implementation
- summarize test result
- summarize review result
- generate traceability matrix
- identify remaining risks and next steps

Main outputs:

- final delivery report
- traceability matrix
- next actions

---

5. Workflow Modes

5.1 Docs-only Mode

Purpose

Generate structured software delivery documents without touching code.

Best For

- early MVP
- requirement clarification
- business analysis
- planning

- users who do not want AI to modify code

Flow

```
Raw requirement
→ BA Agent
→ Product Owner Agent
→ Architect Agent
→ Project Manager Agent
→ QA Agent
→ Reporter Agent
→ Final docs
```

Outputs

- clarified requirement
- acceptance criteria
- scope definition
- technical design
- task breakdown
- test matrix
- final report

No Code Actions

Docs-only mode must not:

- modify source code
- run AI coding agent
- run tests
- create commits
- update external tools automatically

5.2 Assisted Mode

Purpose

Generate high-quality implementation prompts and instructions for AI coding tools.

Best For

- users who manually run Claude Code, Codex CLI, Gemini CLI, Aider, or Cursor
- safer workflows
- users who want control over code execution

Flow

```
Raw requirement
→ Docs generation
→ Technical plan
→ Test matrix
→ Implementation prompt
→ User manually runs coding agent
→ User can paste result back for review
```

Outputs

- all docs-only outputs
- implementation prompt
- coding checklist
- likely changed files
- manual test checklist

Code Actions

The product does not run the coding agent automatically in this mode.

5.3 Semi-autonomous Mode

Purpose

Let the product call one AI coding CLI after user approval.

Best For

- small features
- solo developers
- controlled local coding
- repeatable implementation flow

Flow

```
Raw requirement
→ Requirement clarification
→ Technical planning
→ Task breakdown
→ Test planning
→ User approval
→ Developer Agent runs selected AI CLI
→ Diff collection
→ Test execution
→ Review
```

```
→ Traceability
→ Final report
```

Outputs

- clarified requirement
- technical design
- task breakdown
- test matrix
- implementation prompt
- code diff
- changed files
- test result
- review report
- traceability matrix
- final report

Approval Gates

Required approval before:

- code generation
- applying changes if patch-based
- rollback
- commit or PR creation

5.4 Multi-agent Mode

Purpose

Coordinate multiple AI roles and possibly multiple AI CLI tools.

Best For

- advanced users
- larger features
- future versions
- workflows where different tools are better at different stages

Example Agent Mapping

```
BA Agent: Gemini CLI
Architect Agent: Claude Code
Developer Agent: Codex CLI
QA Agent: Codex CLI
Reviewer Agent: Claude Code
Reporter Agent: Gemini CLI
```

Flow

```
Raw requirement
→ BA Agent
→ Product Owner Agent
→ Project Manager Agent
→ Architect Agent
→ User approval
→ Developer Agent
→ QA Agent
→ Reviewer Agent
→ Security Reviewer Agent
→ Fix loop if needed
→ Reporter Agent
```

Key Rule

Multi-agent mode should not mean uncontrolled parallel execution.

The orchestrator should control:

- sequence
- context passed between agents
- approval gates
- retries
- max iterations
- final decision

5.5 Team Tool Mode

Purpose

Connect the local workflow to external tools such as Jira, GitHub, Slack, Confluence, or Notion.

Best For

- future team workflow
- professional delivery
- small software teams
- freelancers reporting to clients

Flow

```
Requirement
→ Docs
→ Jira-ready tasks or Jira creation
→ Coding workflow
→ GitHub PR summary
```

```
→ Slack progress update
→ Final report
```

Important Rule

External updates should require approval by default.

The product should not silently update Jira, Slack, GitHub, or documentation tools.

---

6. Main End-to-End Workflow

6.1 Workflow Name

Requirement to Delivery Workflow

6.2 Workflow Trigger

The workflow can be triggered from:

- local web UI
- CLI command
- pasted requirement
- existing local run
- future Jira ticket
- future Slack message

6.3 Standard Flow

```
1. Intake
2. Requirement clarification
3. Scope definition
4. Technical planning
5. Task breakdown
6. Test planning
7. User approval
8. Implementation
9. Test execution
10. Review
11. Fix loop
12. Traceability generation
13. Final report
14. Optional external update
```

---

7. Detailed Workflow Stages

7.1 Stage 1: Intake

Purpose

Capture the raw requirement and create a new workflow run.

Input

- raw requirement text
- selected project
- selected workflow mode
- optional project context
- optional language preference
- optional output format

Example Input

```
Thêm API export invoice CSV theo hotelId, date range, status.
```

Actions

The system should:

1. Create a new run.
2. Save raw input.
3. Detect language.
4. Create a run title.
5. Select default workflow mode.
6. Load project settings.
7. Load agent role settings.
8. Load prompt templates.

Output Artifacts

- input.md
- run metadata

Possible Status

- CREATED
- INTAKE_COMPLETED

7.2 Stage 2: Requirement Clarification

Responsible Agent

BA Agent

Purpose

Turn rough input into a clearer requirement.

Input

- raw requirement
- project context if available
- user language preference
- previous run context if available

Actions

The BA Agent should identify:

- business goal
- users/actors
- main flow
- business rules
- edge cases
- validation rules
- permission rules
- open questions
- assumptions
- acceptance criteria

Output Artifacts

- clarified-requirement.md
- business-rules.md
- acceptance-criteria.md
- open-questions.md
- assumptions.md

Quality Checklist

The output is acceptable if:

- the requirement is clearer than raw input
- acceptance criteria are testable
- assumptions are explicitly marked
- unclear points are listed as open questions
- business rules are separated from technical design

User Actions

The user can:

- approve
- edit
- regenerate
- answer open questions
- continue with assumptions

Possible Status

- SPEC_GENERATING
- SPEC_GENERATED
- WAITING_FOR_CLARIFICATION
- SPEC_APPROVED

7.3 Stage 3: Scope Definition

Responsible Agent

Product Owner Agent

Purpose

Define what belongs in the current delivery scope.

Input

- clarified requirement
- acceptance criteria
- open questions
- assumptions

Actions

The Product Owner Agent should define:

- MVP scope
- out-of-scope items
- priority
- success criteria
- release goal
- constraints

Output Artifacts

- scope-definition.md
- success-criteria.md
- out-of-scope.md

Quality Checklist

The output is acceptable if:

- MVP scope is clear
- out-of-scope items are explicit
- priorities are reasonable
- success criteria are measurable

User Actions

The user can:

- approve scope
- add/remove scope
- change priority
- continue

Possible Status

- SCOPE_GENERATED
- SCOPE_APPROVED

7.4 Stage 4: Repository Context Analysis

Responsible Agent

System + Architect Agent

Purpose

Understand the existing project context before design and coding.

Applies To

- brownfield projects
- semi-autonomous mode
- multi-agent mode
- assisted mode with local repo

Input

- selected project path
- repo files
- project settings
- clarified requirement

Actions

The system should detect:

- project type
- language
- framework
- build tool
- test framework
- source folders
- test folders
- migration pattern
- existing conventions
- likely related files/modules

The Architect Agent can then summarize:

- affected modules
- existing patterns
- likely implementation areas
- constraints

Output Artifacts

- repository-analysis.md
- project-summary.json
- affected-modules.md
- existing-patterns.md

Quality Checklist

The output is acceptable if:

- detected stack is correct
- source/test folders are identified
- likely affected modules are reasonable
- project constraints are clear

User Actions

The user can:

- approve
- correct project type
- select affected modules manually
- continue without repo analysis

Possible Status

- REPO_ANALYZING
- REPO_ANALYZED

7.5 Stage 5: Technical Planning

Responsible Agent

Architect Agent

Purpose

Create a technical design before coding.

Input

- clarified requirement
- acceptance criteria
- scope definition
- repository analysis
- existing project conventions

Actions

The Architect Agent should create:

- technical design
- API design
- database design
- service flow
- error handling
- permission model
- integration impact
- risks
- implementation plan

Output Artifacts

- technical-design.md
- api-design.md
- db-design.md
- service-flow.md
- risk-analysis.md
- implementation-plan.md

Quality Checklist

The output is acceptable if:

- design maps to acceptance criteria
- APIs are clearly defined if relevant
- DB impact is described if relevant
- risks are explicit
- affected modules are listed
- implementation plan is actionable

User Actions

The user can:

- approve
- edit design
- regenerate design
- request simpler design
- request more detailed design

Possible Status

- PLAN_GENERATING
- PLAN_GENERATED
- PLAN_APPROVED

7.6 Stage 6: Task Breakdown

Responsible Agent

Project Manager Agent

Purpose

Break work into trackable tasks.

Input

- clarified requirement
- technical design
- implementation plan
- acceptance criteria

Actions

The Project Manager Agent should create:

- epic
- user stories
- subtasks
- dependencies
- priority
- complexity
- definition of done
- Jira-ready task format

Output Artifacts

- task-breakdown.md
- task-breakdown.json

- jira-ready-tasks.md

Quality Checklist

The output is acceptable if:

- tasks are small enough to implement
- each task has acceptance criteria
- dependencies are clear
- tasks map to requirement/design
- output can be copied to Jira or another tool

User Actions

The user can:

- approve
- edit tasks
- merge tasks
- split tasks
- export tasks
- create Jira tasks later

Possible Status

- TASKS_GENERATING
- TASKS_GENERATED
- TASKS_APPROVED

7.7 Stage 7: Test Planning

Responsible Agent

QA Agent

Purpose

Create test scenarios before or alongside implementation.

Input

- clarified requirement
- acceptance criteria
- technical design
- task breakdown

Actions

The QA Agent should create:

- test matrix
- unit test scenarios
- integration test scenarios
- manual test checklist
- permission test cases
- edge cases
- negative cases

Output Artifacts

- test-matrix.md
- test-matrix.json
- manual-test-checklist.md

Quality Checklist

The output is acceptable if:

- each major acceptance criterion has test coverage
- edge cases are included
- negative cases are included
- permission/security cases are included if relevant
- expected results are clear

User Actions

The user can:

- approve
- edit test cases
- add missing cases
- mark tests as manual/automated

Possible Status

- TEST_PLAN_GENERATING
- TEST_PLAN_GENERATED
- TEST_PLAN_APPROVED

7.8 Stage 8: Approval Before Implementation

Responsible Actor

Human User

Purpose

Prevent AI from coding before the plan is accepted.

Input

- clarified requirement
- technical design
- task breakdown
- test matrix
- implementation plan

Approval Screen Should Show

- requirement summary
- assumptions
- open questions
- design summary
- affected modules
- tasks
- test coverage
- risks
- selected AI coding agent
- estimated workflow steps

User Actions

The user can:

- approve implementation
- reject implementation
- edit plan
- go back to previous stage
- switch to assisted mode
- cancel workflow

Output

- approval decision
- approval note if any

Possible Status

- WAITING_FOR_APPROVAL
- APPROVED_FOR_CODE
- IMPLEMENTATION_REJECTED

7.9 Stage 9: Implementation Prompt Generation

Responsible Agent

Developer Agent

Purpose

Generate a high-quality coding prompt for selected AI CLI.

Input

- approved requirement
- approved technical design
- approved task breakdown
- approved test matrix
- coding conventions
- repository context

Actions

The Developer Agent should create an implementation prompt that includes:

- goal
- context
- constraints
- affected areas
- coding rules
- expected tests
- expected output
- forbidden actions
- stop conditions

Output Artifacts

- implementation-prompt.md
- coding-checklist.md

Quality Checklist

The prompt is acceptable if:

- it is specific
- it includes acceptance criteria
- it includes test expectations
- it limits unrelated changes
- it includes file safety rules
- it tells the coding agent when to stop

Possible Status

- IMPLEMENTATION_PROMPT_GENERATED

7.10 Stage 10: Code Execution

Responsible Agent

Developer Agent

Purpose

Use selected AI coding CLI to implement the approved plan.

Applies To

- semi-autonomous mode
- multi-agent mode

Input

- implementation prompt
- project path
- selected AI CLI
- safety rules
- timeout
- max iterations

Actions

The system should:

1. Save git snapshot.
2. Run selected AI CLI.
3. Stream logs.
4. Enforce timeout.
5. Detect command failure.
6. Collect changed files.
7. Generate diff patch.
8. Save implementation notes.

Output Artifacts

- agent-output.log
- changed-files.json
- diff.patch
- implementation-notes.md

Safety Rules

The system should prevent or warn about:

- editing protected files
- running dangerous commands

- exposing secrets
- modifying unrelated files
- modifying production config
- deleting files unexpectedly

Possible Status

- CODING
- CODE_GENERATED
- CODE_FAILED
- PROTECTED_FILE_MODIFIED
- DANGEROUS_COMMAND_BLOCKED

7.11 Stage 11: Test Execution

Responsible Agent

System + QA Agent

Purpose

Verify generated code using configured commands.

Input

- test commands
- changed code
- test matrix
- project path

Actions

The system should run configured commands such as:

- build
- unit test
- integration test
- lint
- type check

The QA Agent should summarize results.

Output Artifacts

- test-result.md
- test-log.txt
- failed-tests.md

Quality Checklist

The output is acceptable if:

- commands executed are listed
- exit codes are included
- pass/fail status is clear
- failed tests are summarized
- possible fix direction is included

Possible Status

- TESTING
- TEST_PASSED
- TEST_FAILED
- TEST_SKIPPED

7.12 Stage 12: Review

Responsible Agents

- Code Reviewer Agent
- Security Reviewer Agent
- QA Agent

Purpose

Review implementation before final delivery.

Input

- clarified requirement
- acceptance criteria
- technical design
- task breakdown
- test matrix
- changed files
- diff
- test result

Actions

The reviewers should check:

- requirement coverage
- code quality
- test coverage
- security risks
- database risks
- unrelated changes

- maintainability
- edge case handling
- production safety

Output Artifacts

- review-report.md
- security-review.md
- requirement-coverage.md

Approval Status

Review result can be:

- APPROVED
- APPROVED_WITH_WARNINGS
- CHANGES_REQUIRED
- REJECTED

Possible Status

- REVIEWING
- REVIEW_PASSED
- REVIEW_FAILED
- REVIEW_CHANGES_REQUIRED

7.13 Stage 13: Fix Loop

Purpose

Allow the coding agent to fix issues found by tests or review.

Applies To

- semi-autonomous mode
- multi-agent mode

Input

- test failures
- review comments
- current diff
- original requirement
- max iteration setting

Flow

```
Test or review fails
→ QA/Reviewer summarizes issue
→ Developer Agent receives fix prompt
```

```
→ AI CLI attempts fix
→ Tests run again
→ Review runs again
→ Stop when passed or max iterations reached
```

Stop Conditions

The fix loop should stop when:

- tests pass and review passes
- max iterations reached
- same error repeats
- dangerous action detected
- user cancels
- user approval required

Output Artifacts

- fix-prompt.md
- fix-agent-output.log
- updated-diff.patch
- iteration-summary.md

Possible Status

- FIXING
- FIX_APPLIED
- FIX_FAILED
- MAX_ITERATIONS_REACHED

7.14 Stage 14: Traceability Generation

Responsible Agent

Reporter Agent

Purpose

Map requirements to output.

Input

- clarified requirement
- acceptance criteria
- task breakdown
- changed files
- test matrix
- test result
- review report

Actions

The Reporter Agent should generate a matrix mapping:

- requirement ID
- acceptance criteria
- task
- code files
- test cases
- test result
- coverage status

Output Artifacts

- traceability.md
- traceability.json

Possible Status

- TRACEABILITY_GENERATED

7.15 Stage 15: Final Report

Responsible Agent

Reporter Agent

Purpose

Summarize the completed workflow.

Input

- all generated artifacts
- test result
- review result
- traceability matrix
- user decisions

Final Report Should Include

- original requirement
- clarified requirement
- assumptions
- open questions
- scope summary
- technical summary
- task summary
- implementation summary
- changed files

- test result
- review result
- traceability summary
- risks
- next steps

Output Artifacts

- final-report.md

Possible Status

- REPORT_GENERATING
- REPORT_GENERATED

7.16 Stage 16: Optional External Update

Responsible Actor

Human User + Integration Agent

Purpose

Publish or sync results to external tools.

Possible External Actions

- create Jira tasks
- comment on Jira ticket
- create GitHub PR
- comment on GitHub PR
- post Slack summary
- publish docs to Confluence/Notion

Required Approval

The product must ask approval before each external update.

Output Artifacts

- external-update-log.md
- integration-result.json

Possible Status

- EXTERNAL_UPDATE_PENDING
- EXTERNAL_UPDATE_COMPLETED
- EXTERNAL_UPDATE_FAILED

---

8. Workflow Status Model

8.1 High-level Statuses

```
CREATED
INTAKE_COMPLETED
SPEC_GENERATING
SPEC_GENERATED
WAITING_FOR_CLARIFICATION
SPEC_APPROVED
SCOPE_GENERATED
SCOPE_APPROVED
REPO_ANALYZING
REPO_ANALYZED
PLAN_GENERATING
PLAN_GENERATED
PLAN_APPROVED
TASKS_GENERATED
TEST_PLAN_GENERATED
WAITING_FOR_APPROVAL
APPROVED_FOR_CODE
CODING
CODE_GENERATED
CODE_FAILED
TESTING
TEST_PASSED
TEST_FAILED
REVIEWING
REVIEW_PASSED
REVIEW_FAILED
FIXING
MAX_ITERATIONS_REACHED
TRACEABILITY_GENERATED
REPORT_GENERATED
EXTERNAL_UPDATE_PENDING
EXTERNAL_UPDATE_COMPLETED
FAILED
CANCELLED
```

8.2 Terminal Statuses

A run is finished when status is one of:

```
REPORT_GENERATED
EXTERNAL_UPDATE_COMPLETED
FAILED
CANCELLED
```

8.3 Partial Success

A run can still be useful even if implementation fails.

Example:

```
Docs generated successfully.
Code generation failed.
Final report generated with failure summary.
```

The product should treat partial artifacts as valuable.

---

9. Approval Gates

9.1 Required Approval Gates

Gate 1: Requirement Approval

Before technical planning, user can approve or edit clarified requirement.

Gate 2: Plan Approval

Before implementation, user must approve:

- technical design
- task breakdown
- test matrix
- implementation plan

Gate 3: Code Generation Approval

Before running AI coding CLI, user must approve execution.

Gate 4: Risky File Approval

If AI modifies sensitive or warning files, user must approve continuation.

Gate 5: External Update Approval

Before updating Jira, Slack, GitHub, Confluence, or Notion, user must approve.

Gate 6: Rollback Approval

Before rollback, user must confirm.

9.2 Optional Approval Gates

Users can configure approval before:

- test execution
- review execution
- report generation
- opening PR
- creating commit

---

10. Workflow Failure Handling

10.1 Failure Types

Requirement Failure

Examples:

- input is too vague
- conflicting requirements
- missing critical business rules

Action:

- ask clarification questions
- allow user to continue with assumptions

AI Agent Failure

Examples:

- AI CLI not installed
- AI CLI exits with error
- prompt too large
- agent stuck

Action:

- show error
- suggest another agent
- allow retry
- save logs

Command Failure

Examples:

- build failed
- tests failed
- timeout

- dangerous command blocked

Action:

- summarize failure
- offer fix loop
- allow user to stop and keep artifacts

Safety Failure

Examples:

- protected file modified
- secret detected in logs
- dangerous command attempted

Action:

- stop workflow
- show warning
- require user decision

Integration Failure

Examples:

- Jira token invalid
- Slack channel missing
- GitHub PR creation failed

Action:

- keep local artifacts
- show integration error
- allow retry after settings update

---

11. Workflow Output Structure

Each run should produce a clear artifact structure.

```
run/
  input.md

  requirement/
    clarified-requirement.md
    business-rules.md
    acceptance-criteria.md
    open-questions.md
    assumptions.md
```

```
scope/
  scope-definition.md
  out-of-scope.md
  success-criteria.md

design/
  repository-analysis.md
  technical-design.md
  api-design.md
  db-design.md
  service-flow.md
  risk-analysis.md

tasks/
  task-breakdown.md
  task-breakdown.json
  jira-ready-tasks.md

tests/
  test-matrix.md
  manual-test-checklist.md
  test-result.md
  failed-tests.md

implementation/
  implementation-plan.md
  implementation-prompt.md
  coding-checklist.md
  agent-output.log
  changed-files.json
  diff.patch

review/
  review-report.md
  security-review.md
  requirement-coverage.md

report/
  final-report.md
  traceability.md
  traceability.json

logs/
  workflow.log
  shell.log
  error.log
```

---

12. Workflow UI Requirements

12.1 Workflow Timeline

The local web UI should show a timeline:

```
Intake → Requirement → Scope → Design → Tasks → Tests → Approval → Code →
Review → Report
```

Each stage should show:

- status
- responsible agent
- generated artifacts
- errors if any
- next action

12.2 Agent Team Room

The UI should show agent messages like a team workspace.

Example:

```
BA Agent:
I found 4 missing business rules. Please confirm permission, date range
validation, export size limit, and empty result behavior.

Architect Agent:
The proposed API is GET /api/invoices/export. It affects InvoiceController,
InvoiceService, and InvoiceRepository.

QA Agent:
I created 8 test cases. 5 normal cases, 2 edge cases, and 1 permission case.

Reviewer Agent:
Review passed with warnings. CSV export needs max row limit to avoid
performance issue.
```

12.3 Run Detail Tabs

Each run should have tabs:

- Input
- Requirement
- Scope
- Design
- Tasks

- Tests
- Implementation
- Review
- Traceability
- Report
- Logs

12.4 Approval Screen

Approval screen should show:

- what will happen next
- which agent will run
- what files may be affected
- what commands may run
- risks
- user options

User options:

- approve
- reject
- edit plan
- switch mode
- cancel

---

13. Workflow CLI Requirements

13.1 CLI Flow Commands

The CLI should support stage-by-stage execution:

```
aiteam run
aiteam spec
aiteam scope
aiteam analyze
aiteam plan
aiteam tasks
aiteam tests
aiteam code
aiteam review
aiteam report
```

13.2 Full Run Command

The full run command should execute the selected mode:

```
aiteam run "raw requirement" --mode docs-only
aiteam run "raw requirement" --mode assisted
aiteam run "raw requirement" --mode semi-auto --agent codex
```

13.3 Resume Command

The CLI should support resuming a failed or paused run:

```
aiteam resume <runId>
```

13.4 Cancel Command

The CLI should support cancellation:

```
aiteam cancel <runId>
```

13.5 Report Command

The CLI should support report generation anytime:

```
aiteam report <runId>
```

---

14. Example Workflow: Docs-only Mode

Input

```
Thêm API export invoice CSV theo hotelId, date range, status.
```

Flow

```
1. Intake
2. BA Agent clarifies requirement
3. PO Agent defines scope
4. Architect Agent creates technical design
5. PM Agent creates task breakdown
6. QA Agent creates test matrix
7. Reporter Agent creates final report
```

Output

```
clarified-requirement.md
acceptance-criteria.md
scope-definition.md
technical-design.md
api-design.md
task-breakdown.md
test-matrix.md
final-report.md
```

User Experience

User gets a complete delivery plan without any code modification.

---

15. Example Workflow: Semi-autonomous Mode

Input

```
Thêm chức năng reset password bằng email OTP.
```

Flow

```
1. Intake
2. BA Agent clarifies requirement
3. Architect Agent creates design
4. PM Agent creates tasks
5. QA Agent creates test matrix
6. User approves implementation
7. Developer Agent runs selected AI CLI
8. System collects diff
9. System runs tests
10. Reviewer Agent reviews implementation
11. Fix loop runs if needed
12. Reporter Agent creates final report
```

Output

```
requirement docs
technical design
task breakdown
test matrix
implementation prompt
```

```
diff.patch
test-result.md
review-report.md
traceability.md
final-report.md
```

User Experience

User gets generated code with review and test summary, but still keeps final control.

---

16. Example Workflow: Assisted Mode

Input

```
Tạo approval flow cho booking cancellation. Nếu refund > 50 triệu thì cần
director duyệt.
```

Flow

```
1. Intake
2. Requirement clarification
3. Technical design
4. Task breakdown
5. Test matrix
6. Implementation prompt generation
7. User manually copies prompt to AI coding tool
8. User can paste result back for review
```

Output

```
implementation-prompt.md
coding-checklist.md
manual-test-checklist.md
```

User Experience

User gets a powerful prompt package for Claude Code, Codex CLI, Gemini CLI, Aider, or another tool.

---

17. Traceability Workflow

Purpose

Traceability should be generated after planning and updated after implementation if available.

Before Code

The system can map:

```
Requirement → Acceptance Criteria → Task → Test Case
```

After Code

The system can map:

```
Requirement → Acceptance Criteria → Task → Code File → Test Case → Test
Result
```

Example

```
REQ-001:
User can export invoices by hotelId.

AC-001:
Export must filter invoices by hotelId.

TASK-001:
Implement invoice export query.

CODE:
InvoiceExportService.java
InvoiceRepository.java

TEST:
InvoiceExportServiceTest.shouldExportInvoicesByHotelId()

STATUS:
Covered
```

---

18. Workflow Quality Gates

18.1 Requirement Quality Gate

Requirement stage passes if:

- requirement is understandable
- user roles are identified
- acceptance criteria exist
- assumptions are listed
- open questions are listed

18.2 Design Quality Gate

Design stage passes if:

- affected modules are listed
- API/DB impact is described if relevant
- risks are listed
- implementation plan is actionable

18.3 Task Quality Gate

Task stage passes if:

- tasks are clear
- each task has acceptance criteria
- dependencies are listed
- definition of done exists

18.4 Test Quality Gate

Test stage passes if:

- each major acceptance criterion has test coverage
- negative cases exist
- edge cases exist
- expected results are clear

18.5 Implementation Quality Gate

Implementation stage passes if:

- changed files are collected
- diff is generated
- no protected file is modified without approval
- no dangerous command is executed
- implementation follows approved plan

18.6 Review Quality Gate

Review stage passes if:

- tests pass or failure is explained
- review approves or lists required fixes
- requirement coverage is acceptable
- risks are documented

---

19. Workflow Configuration

Users should be able to configure:

- default workflow mode
- default language
- default output language
- default AI agent per role
- approval gates
- max iterations
- test commands
- protected files
- documentation outputs
- integration behavior

Example configuration options:

```
Default mode: docs-only
Default output language: English
Requirement input language: Vietnamese or English
Require approval before code: true
Max fix iterations: 3
Generate traceability: true
Generate Jira-ready tasks: true
```

---

20. Workflow Roadmap

Phase 1: Docs-only Workflow

Goal:

```
Rough requirement → clear docs and task plan
```

Includes:

- intake

- requirement clarification
- scope
- design
- task breakdown
- test matrix
- final report

Phase 2: Assisted Workflow

Goal:

```
Docs → implementation prompt package
```

Includes:

- implementation prompt
- coding checklist
- expected changed files
- manual test checklist

Phase 3: Semi-autonomous Workflow

Goal:

```
Docs → code patch → test → review → report
```

Includes:

- AI CLI execution
- diff collection
- test runner
- review report
- traceability update

Phase 4: Multi-agent Workflow

Goal:

```
Different AI agents for different roles
```

Includes:

- role-to-agent mapping
- multi-agent handoff
- fix loop
- stronger review

Phase 5: Team Tool Workflow

Goal:

```
Local AI team connects to real team tools
```

Includes:

- GitHub PR
- Jira tasks
- Slack progress
- Confluence/Notion docs

---

21. Final Workflow Summary

Local AI Software Team should not simply send a prompt to a coding agent.

It should guide a software delivery workflow:

```
Raw requirement
→ Clarified requirement
→ Scope
→ Technical design
→ Task breakdown
→ Test matrix
→ Approval
→ Implementation
→ Test
→ Review
→ Traceability
→ Final report
```

The core value is not only coding.

The core value is controlled software delivery from vague input to reviewable output.

The product should start with docs-only and assisted workflows, then gradually add code execution, review loops, traceability, and optional team tool integrations.